ELEC3104: Digital Signal Processing Lab

Topic 5: Digital Filter Design

MATLAB Examples

Learning Objectives

By the end of this topic, you should be able to:

- Transform an analogue filter into a digital filter using the **impulse invariance method**.
- Determine the **order of an IIR filter** from given specifications.
- Design **Butterworth, Chebyshev Type I, Chebyshev Type II and Elliptic IIR filters** using MATLAB.
- Design **linear-phase FIR filters** using the window method.
- Plot and interpret the **magnitude, phase and group-delay responses** of digital filters.
- Apply digital filtering to practical signals.

Prof. Eliathamby Ambikairajah, School of EE&T

Term 3, 2026

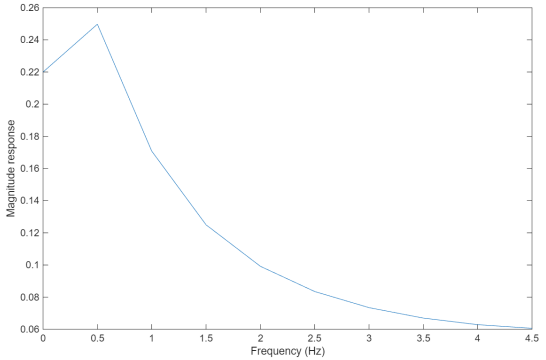
Topic 5 – Digital Filter Design

Example 1 - Impulse Invariance

Consider the Analogue filter H(s) given by : $H(s) = \frac{s+1}{s^2+5s+6}$

1. Transform H(s) into a digital filter H(z) using the impulse invariance method. (The sampling period T =0.1 sec)
2. Use the MATLAB function " impinvar" to obtain H(z).
3. Plot the magnitude response of the resulting digital filter.

```
b = [1 1]; % nominator coef.
a = [1 5 6]; % denominator coef.
T = 0.1; % sampling period
fs = 1/T; % sampling frequency
[bz, az] = impinvar(b, a, fs); % H(z)
[H, w] = freqz(bz, az, fs);
figure
plot((fs/2)*w/pi, abs(H));
xlabel('Frequency (Hz)');
ylabel('Magnitude response');
```

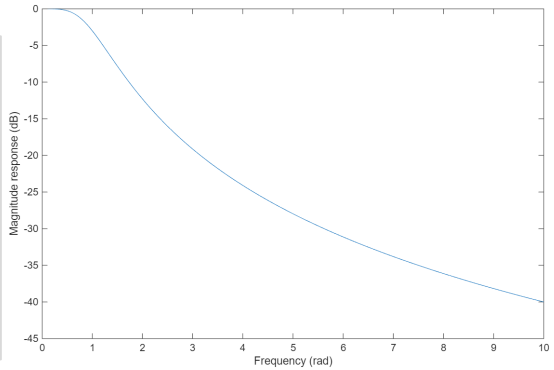


Example 2 - Butterworth Low-Pass Filter

The Transfer function $G(s) = \frac{1}{s^2+\sqrt{2}s+1}$ represents a normalised second-order Butterworth low-pass filter

1. Determine the frequency at which the magnitude response falls by 3 dB.
2. Use MATLAB to plot the magnitude response.
3. Verify the calculated 3-dB cut-off frequency from the MATLAB result.

```
b = [1]; % nominator coef.
a = [1 sqrt(2) 1]; % denominator coef.
[H, w] = freqs(b, a);
figure
plot(w, 20*log10(abs(H)));
xlabel('Frequency (rad)');
ylabel('Magnitude response (dB)');
```



Topic 5 – MATLAB Commands for Digital Filter Design

Estimation of Order of IIR Filter

The first step in the filter design process is to choose the type of filter approximation to be employed and then to estimate the order of the transfer function from the filter specifications. The MATLAB commands for estimating the order of a filter are:

1. Butterworth:

```
[N, ωn] = buttord(ωp, ωs, rp, rs)
```

2. Chebyshev 1:

```
[N, ωn] = cheb1ord(ωp, ωs, rp, rs)
```

3. Chebyshev 2:

```
[N, ωn] = cheb2ord(ωp, ωs, rp, rs)
```

4. Elliptic:

```
[N, ωn] = ellipord(ωp, ωs, rp, rs)
```

IIR Filter Design

After the filter type has been selected and its order estimated, the next step is to determine the transfer function of the filter. To this end, MATLAB provides functions for all four types of filters.

For designing a **Butterworth digital low-pass or band-pass filter:**

```
[num, dem] = butter(N, ωn)
```

For designing a **Butterworth digital high-pass filter**:

```
[num, dem] = butter(N, ωn, 'high')
```

The following command returns the transfer function of a **Butterworth band-stop filter**:

```
[num, dem] = butter(N, ωn, 'stop')
```

Chebyshev Type I:

```
[num, dem] = cheby1(N, rp, ωn)
[num, dem] = cheby1(N, rp, ωn, 'filtertype')
```

Chebyshev Type II:

```
[num, dem] = cheby2(N, rs, ωn)
[num, dem] = cheby2(N, rs, ωn, 'filtertype')
```

Elliptic:

```
[num, dem] = ellip(N, rp, rs, ωn)
[num, dem] = ellip(N, rp, rs, ωn, 'filtertype')
```

Topic 5 – IIR Digital Filter Design

Example 3 – IIR Filter Order

(a) Design a digital **band-pass IIR filter** with the following specifications:

- Sampling frequency: fs=7 kHz
- Passband: **1.4–2.1 kHz**
- Stopband edges: **1.05 kHz and 2.45 kHz**
- Maximum passband ripple: **0.4 dB**
- Minimum stopband attenuation: **50 dB**

(b) Using MATLAB, determine the minimum filter order for:

- Butterworth
- Chebyshev Type I
- Chebyshev Type II
- Elliptic

(b) Compare the required filter orders and discuss your results.

(b) Using the given specifications, design an elliptic digital IIR band-pass filter.

- Determine the required filter order;
- Obtain the transfer function H(z);
- Plot magnitude, unwrapped phase and group-delay;
- Verify that the designed filter satisfies the given specifications.

```
Fs=7000; % sampling frequency
Rp = 0.4; % max passband ripple
Rs = 50; % min. stopband attenuation
Wp = [1400 2100]/(fs/2); % passband (normalized)
Ws = [1050 2450]/(fs/2); % stopband edges (normalized)
[Nb, Wnb] = buttord(Wp, Ws, Rp, Rs); % Butterworth
[Nc1, Wnc1] = cheb1ord(Wp, Ws, Rp, Rs); % Chebyshev 1
[Nc2, Wnc2] = cheb2ord(Wp, Ws, Rp, Rs); % Chebyshev 2
[Ne, Wne] = ellipord(Wp, Ws, Rp, Rs); % Elliptic

% transfer function of elliptic
[numE, demE] = ellip(Ne, Rp, Rs, Wne);
[H, w] = freqz(numE, demE);
figure
plot(w/pi*(fs/2), 20*log10(abs(H))); % plot magnitude response
xlabel('Frequency (Hz)');
ylabel('Magnitude response (dB)');

figure
plot(w/pi*(fs/2), unwrap(angle(H))); % plot unwrapped phase
xlabel('Frequency (Hz)');
ylabel('Phase response');

[gd, wgd] = grpdelay(numE, demE); % Group delay
figure
plot(wgd/pi*(fs/2), gd);
xlabel('Frequency (Hz)'); ylabel('Group Delay (samples)');
```

Topic 5 – FIR Filter Design

Example 4 – FIR Filter Design

Design a linear-phase FIR low-pass filter with: $f_{pass} = 2\ kHz$, $f_{stop} = 2.5\ kHz$, Sampling frequency: $f_s = 16\ kHz$. Passband and stopband ripple: $\delta p = \delta s = 0.005$.

1. Use kaiserord to estimate the required filter order.
2. Design the FIR filter.
3. Plot its magnitude and phase responses.
4. Display the filter coefficients.
5. Check whether the filter satisfies the specifications.
6. If necessary, increase the filter order until the specifications are satisfied.

```
fs = 16000; % sampling frequency
% Estimate filter order
[N, Wn, bta, filtype] = kaiserord([2000 2500], [1 0], [0.005 0.005], fs);
fprintf('Initial filter order from kaiserord = %d\n', N);
B = fir1(N, Wn); % design filter
[H, w] = freqz(B, 1); % frequency response
figure
subplot 211
plot(w/pi*(fs/2), abs(H)); % plot magnitude response
ylabel('Magnitude response');
subplot 212
plot(w/pi*(fs/2), unwrap(angle(H))); % plot unwrapped phase
ylabel('Phase response');  xlabel('Frequency (Hz)');
```

FIR Filter Design

The simplest approach to FIR filter design is to simply truncate (apply a finite-length window) an infinite impulse response to a finite number of terms. The functions `fir1` and `fir2` can be employed to design windowed FIR digital filters in MATLAB. Both functions yield a linear-phase design.

The function `fir1` can be used to design conventional low-pass, high-pass, band-pass and band-stop linear-phase FIR filters. The command is:

$$b = \text{fir1}(N, \omega_n)$$

The above command returns a vector b, the impulse response coefficients, arranged in ascending powers of z^{-1}, of a low-pass filter of order N. The command

$$b = \text{fir1}(N, \omega_n, \text{'high'})$$

is used to design a high-pass filter with N an even integer. The Hamming window is employed as a default. If a different window is to be used, then the command is:

$$\text{taper} = \text{hanning}(N);$$

$$b = \text{fir1}(N, \omega_n, \text{taper})$$

Topic 5 – FIR Filter Design

Example 5 – Comparing FIR and IIR Filters

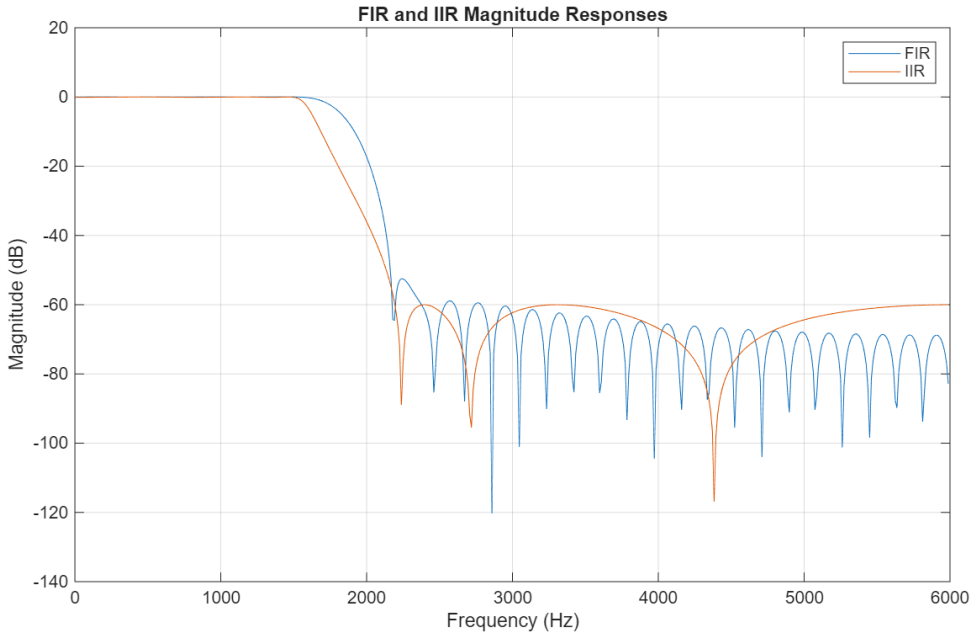
Design an FIR and an IIR low-pass filter to approximately satisfy the same frequency specifications. Using MATLAB:

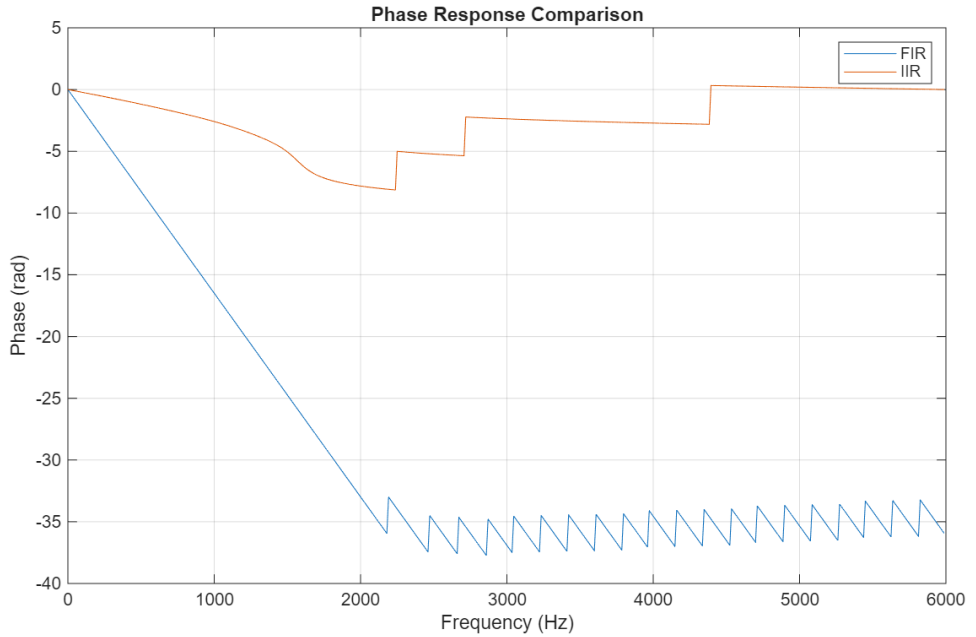
- Plot the magnitude responses of both filters on the same figure.
- Plot and compare their phase responses and group delays.
- Compare the number of filter coefficients required.
- Which filter has the more linear phase response?
- Which filter requires fewer coefficients to achieve a similar magnitude response?
- Briefly discuss the advantages and disadvantages of the FIR and IIR implementations.

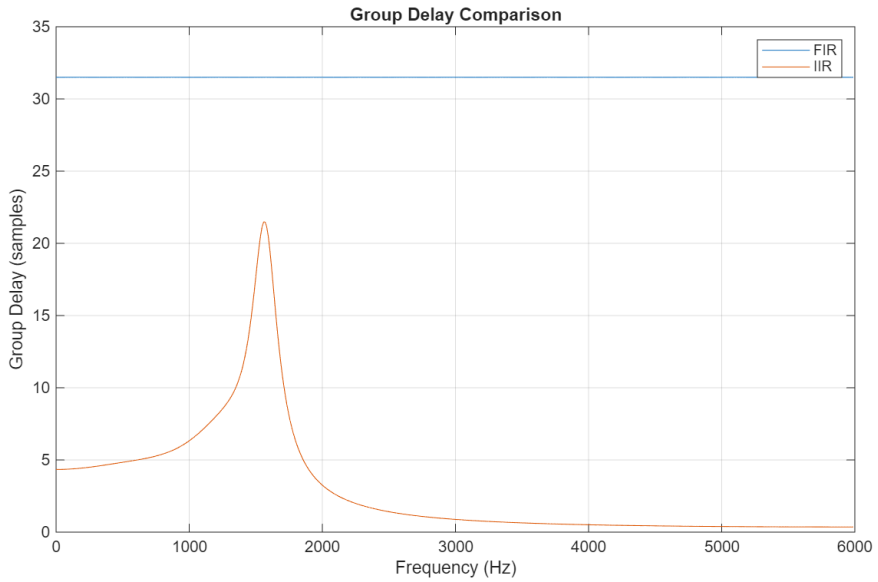
```
fs = 12000; % sampling frequency
fpass = 1500; % passband frquency
fstop = 2200; % stopbandfrequency
dp = 0.01; % passband ripple
ds = 0.001; % stopband ripple
% FIR FILTER
[Nfir, Wnfir, beta, filtype] = kaiserord([fpass fstop], [1 0], [dp ds], fs);
Bfir = fir1(Nfir, Wnfir);
fprintf('FIR filter order = %d\n', Nfir);
fprintf('Number of FIR coefficients = %d\n', length(Bfir));
```

```
% IIR FILTER - ELLIPTIC
Rp = -20*log10(1-dp); % convert passband ripples into dB
Rs = -20*log10(ds); % convert stopband ripples into dB
Wp = fpass/(fs/2); % passband (normalized)
Ws = fstop/(fs/2); % stopband (normalized)
[Niir, Wniir] = ellipord(Wp, Ws, Rp, Rs);
[Biir, Aiir] = ellip(Niir, Rp, Rs, Wniir);
fprintf('\nIIR filter order = %d\n', Niir);
fprintf('Numerator coefficients = %d\n', length(Biir));
fprintf('Denominator coefficients = %d\n', length(Aiir));
% FREQUENCY RESPONSES
[Hfir, wfir] = freqz(Bfir, 1); % FIR
[Hiir, wiir] = freqz(Biir, Aiir); % IIR
% Magnitude responses
figure;
plot(wfir/pi*(fs/2), 20*log10(abs(Hfir)));
hold on;
plot(wiir/pi*(fs/2), 20*log10(abs(Hiir)));
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('FIR and IIR Magnitude Responses'); legend('FIR', 'IIR'); grid on;
% Phase responses
figure;
plot(wfir/pi*(fs/2), unwrap(angle(Hfir)));
hold on;
plot(wiir/pi*(fs/2), unwrap(angle(Hiir)));
xlabel('Frequency (Hz)'); ylabel('Phase (rad)');
title('Phase Response Comparison'); legend('FIR', 'IIR'); grid on;
% Group delay
[gdfir, wgdfir] = grpdelay(Bfir, 1); % FIR
[gdiir, wgdiir] = grpdelay(Biir, Aiir); % IIR
figure;
plot(wgdfir/pi*(fs/2), gdfir);
hold on;
plot(wgdiir/pi*(fs/2), gdiir);
xlabel('Frequency (Hz)'); ylabel('Group Delay (samples)');
title('Group Delay Comparison'); legend('FIR', 'IIR'); grid on;
```

Topic 5 – FIR Filter Design







Topic 5 – FIR Filter Design

Example 6 – Effect of Window Functions on FIR Filter Design

Design the same FIR low-pass filter using:

- Rectangular window
- Hanning window
- Hamming window

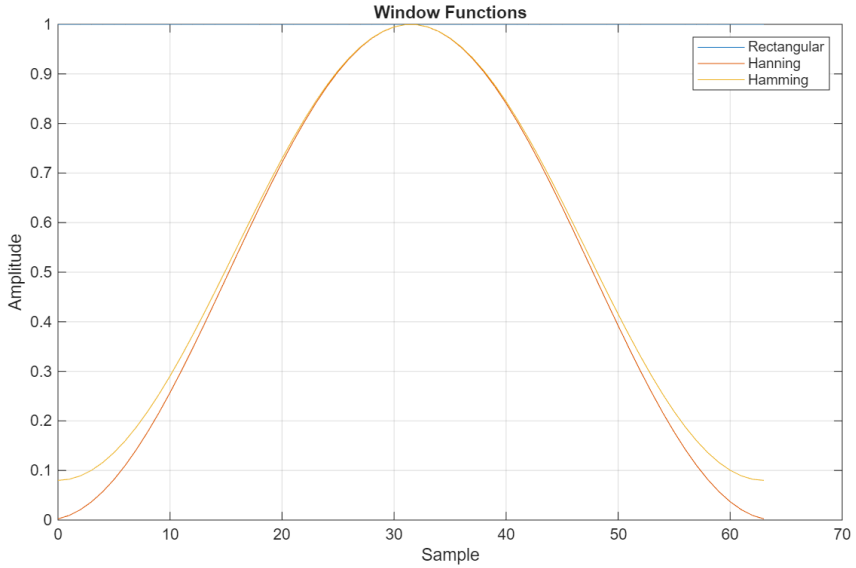
Using MATLAB:

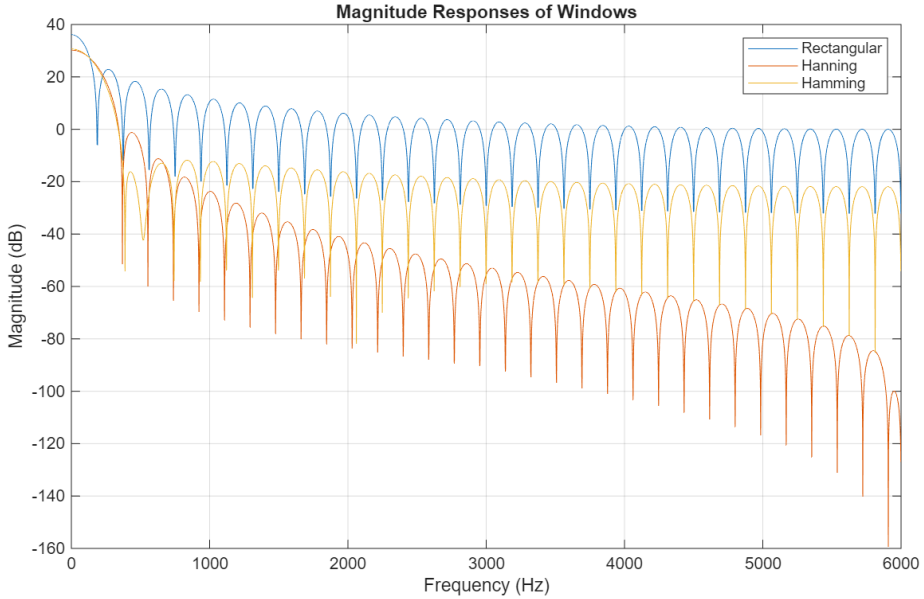
- Plot the three window functions.
- Plot their magnitude responses.
- Design the FIR low-pass filter using each window.
- Compare the resulting filter magnitude responses.
- Comment on the main-lobe width, side-lobe level, transition width and stopband attenuation.
- Which window would you choose for this filter, and why?

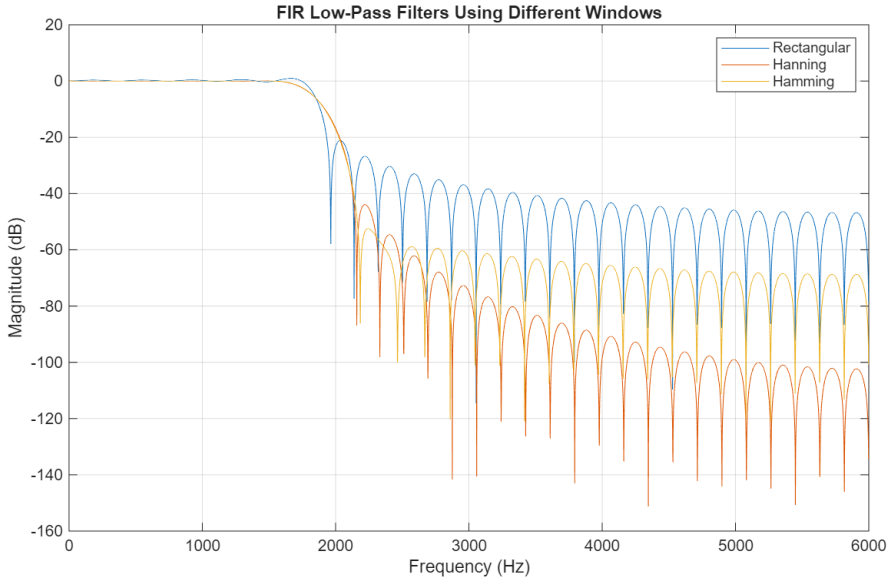
```
[N, Wn, beta, filtype] = kaiserord([fpass fstop], [1 0], [dp ds], fs); % FIR
% window functions
w_rect = rectwin(N+1); % rectangular window
w_hann = hanning(N+1); % Hanning window
w_hamm = hamming(N+1); % Hamming window
figure;
plot(0:N, w_rect); % plot rectangular window
hold on;
plot(0:N, w_hann); % plot Hanning window
plot(0:N, w_hamm); % plot Hamming window
xlabel('Sample'); ylabel('Amplitude'); title('Window Functions');
legend('Rectangular', 'Hanning', 'Hamming'); grid on;
```

```
% Frquency responses
Nfft = 4096;
[Hr, fr] = freqz(w_rect, 1, Nfft, fs);
[Hn, ~] = freqz(w_hann, 1, Nfft, fs);
[Hm, ~] = freqz(w_hamm, 1, Nfft, fs);
% plot Magnitude responses
figure;
plot(fr, 20*log10(abs(Hr))); % rectangular window
hold on;
plot(fr, 20*log10(abs(Hn))); % Hanning window
plot(fr, 20*log10(abs(Hm))); % Hamming window
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
title('Magnitude Responses of Windows');
legend('Rectangular', 'Hanning', 'Hamming'); grid on;
% FIR FILTER DESIGN USING EACH WINDOW
b_rect = fir1(N, Wn, w_rect);
b_hann = fir1(N, Wn, w_hann);
b_hamm = fir1(N, Wn, w_hamm);
% Frequency responses
[Hrect, f] = freqz(b_rect, 1, Nfft, fs); % rectangular window
[Hhann, ~] = freqz(b_hann, 1, Nfft, fs); % Hanning window
[Hhamm, ~] = freqz(b_hamm, 1, Nfft, fs); % Hamming window
figure;
plot(f, 20*log10(abs(Hrect)));
hold on;
plot(f, 20*log10(abs(Hhann)));
plot(f, 20*log10(abs(Hhamm)));
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
title('FIR Low-Pass Filters Using Different Windows'); legend('Rectangular',
'Hanning', 'Hamming'); grid on;
```

Topic 5 – FIR Filter Design







Topic 5 – Speech filtering exercise - Importance of the Fundamental Frequency

Background

Speech sounds can be broadly classified as voiced and unvoiced. Voiced speech is produced when the vocal cords vibrate, producing a fundamental frequency, also referred to as the pitch frequency. The fundamental frequency is typically in the range of approximately 50–400 Hz, depending on the speaker. The vocal tract also produces a number of resonant frequencies called formants. Several formants are present below 4 kHz and play an important role in the spectral characteristics of speech. In this exercise , you will investigate experimentally what happens to speech when the fundamental-frequency region is removed.

Part A – Record and Analyse Your Speech

Record approximately 5–10 seconds of your own speech at a sampling frequency of 16 kHz and save it as a .wav file.
Using MATLAB:

- Read and play your recorded speech.
- Plot the speech waveform.
- Plot the magnitude spectrum with the frequency axis in Hz.
- From the spectrum, observe the frequency components present in your speech.

Part B – Design a Digital Filter

Design a digital filter of your choice to remove the frequency components below approximately 400 Hz, while retaining the higher-frequency components of the speech signal. You should:

- Select a suitable filter type.
- Choose appropriate filter specifications.
- Determine the required filter order.
- Plot the magnitude response of your designed filter.
- Explain and justify your filter design.

Part C – Filter the Speech

Apply your designed filter to the recorded speech. Plot and compare the magnitude spectra of:

- the original speech signal, and
- the filtered speech signal.
- Use the spectra to demonstrate that the fundamental-frequency region has been significantly attenuated.

Topic 5 – Speech filtering exercise - Importance of the Fundamental Frequency

Part D – Listen and Investigate

Listen carefully to:

- the **original speech**, with the fundamental-frequency region present; and
- the **filtered speech**, with the fundamental-frequency region removed.

Compare the two signals in terms of **speech intelligibility, perceived pitch and speech quality**.

Based on your listening observations and spectral analysis, answer the following question:

How important is the fundamental frequency for understanding speech? What changes do you observe when the fundamental-frequency region is removed?

```
% Part a
[x, fs] = audioread('speech.wav'); % Read speech file
fprintf('Sampling frequency = %d Hz\n', fs);
fprintf('Duration = %.2f seconds\n', length(x)/fs);
sound(x, fs); % Play speech
% Time axis
N = length(x);
t = (0:N-1)/fs;
% Plot speech waveform
figure;
plot(t, x);
xlabel('Time (s)'); ylabel('Amplitude'); title('Recorded Speech Waveform');
grid on;
X = fft(x);
Xmag = abs(X);
f = (0:N-1)*fs/N;
figure;
plot(f, Xmag); xlabel('Frequency (Hz)'); ylabel('Magnitude');
title('Magnitude Spectrum of Original Speech');
```

```
% Part b
Fp = 500; % Passband edge frequency (Hz)
Fst = 300; % Stopband edge frequency (Hz)
Rp = 1; % Maximum passband ripple (dB)
Rs = 40; % Minimum stopband attenuation (dB)
% Normalised frequencies
Wp = Fp/(fs/2);
Ws = Fst/(fs/2);
% Determine minimum Butterworth filter order
[n, Wn] = buttord(Wp, Ws, Rp, Rs);
fprintf('\nButterworth high-pass filter\n');
fprintf('Filter order = %d\n', n);
fprintf('Normalised cutoff frequency = %.4f\n', Wn);
% Design Butterworth high-pass filter
[b, a] = butter(n, Wn, 'high');
% Plot magnitude response
figure;
freqz(b, a, 2048, fs);
title(['Butterworth High-Pass Filter, Order = ', num2str(n)]);
```

```
% Part c
y = filter(b, a, x);
sound(y, fs); % filtered signal
Y = fft(y);
Ymag = abs(Y);
figure;
subplot(2,1,1);
plot(f, Xmag);
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Original Speech Spectrum');
subplot(2,1,2);
plot(f, Ymag);
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Filtered Speech Spectrum');
```

